SOFTWARE ENGINEERING LAB 3

Component Modelling & Architectural Pattern Selection

Scenario: Self-Service Coffee Kiosk System

Student Name: _____Keertika Acharya_____ SRN: _____PES1UG24AM133_____ Section: _____C_____

1. Objective

Evaluate different architectural styles, select the most appropriate style for the assigned scenario, and create a UML component diagram showing modules, interfaces, and dependencies.

2. Scenario and Requirements

- Customers can select Espresso, Americano, or Latte.
- Customers can choose Small or Large.
- Payment is by credit card only.
- The system prints receipts containing order details.
- The system must support touchscreen interactions, connect to receipt-printer hardware, and store menu and pricing information.

3. Architectural Style Analysis

Architecture	Advantages	Disadvantages
Layered	Separates presentation, business logic, and data; clear and easy to understand.	Layer transitions can add overhead; scaling individual components may be difficult.
Microservices	Independent deployment and scaling, fault isolation, and technology diversity.	Operational complexity, network latency, and data-consistency challenges.
Client-Server	Centralized control, simple deployment, and easier centralized data consistency.	Possible server bottleneck and single point of failure.

4. Selected Architecture and Justification

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Reason 1 — Separation of concerns

The User Interface handles touchscreen interactions, the Order Manager validates and coordinates orders, the Payment Service processes credit-card payments, and the Menu & Pricing Database stores menu information. This division makes components easier to understand, test, and maintain.

Reason 2 — Suitability for the workflow

The kiosk has a small, well-defined sequence: select a coffee and size, pay, and receive a receipt. Layered Architecture supports this workflow without the operational complexity of microservices.

Security Advantage

The interface is separated from payment-processing logic. The Order Manager validates orders and coordinates requests, while the Payment Service handles payment processing. Card data should be handled by a secure payment provider or terminal, using secure communication and avoiding unnecessary storage of sensitive card details.

Performance Benefit

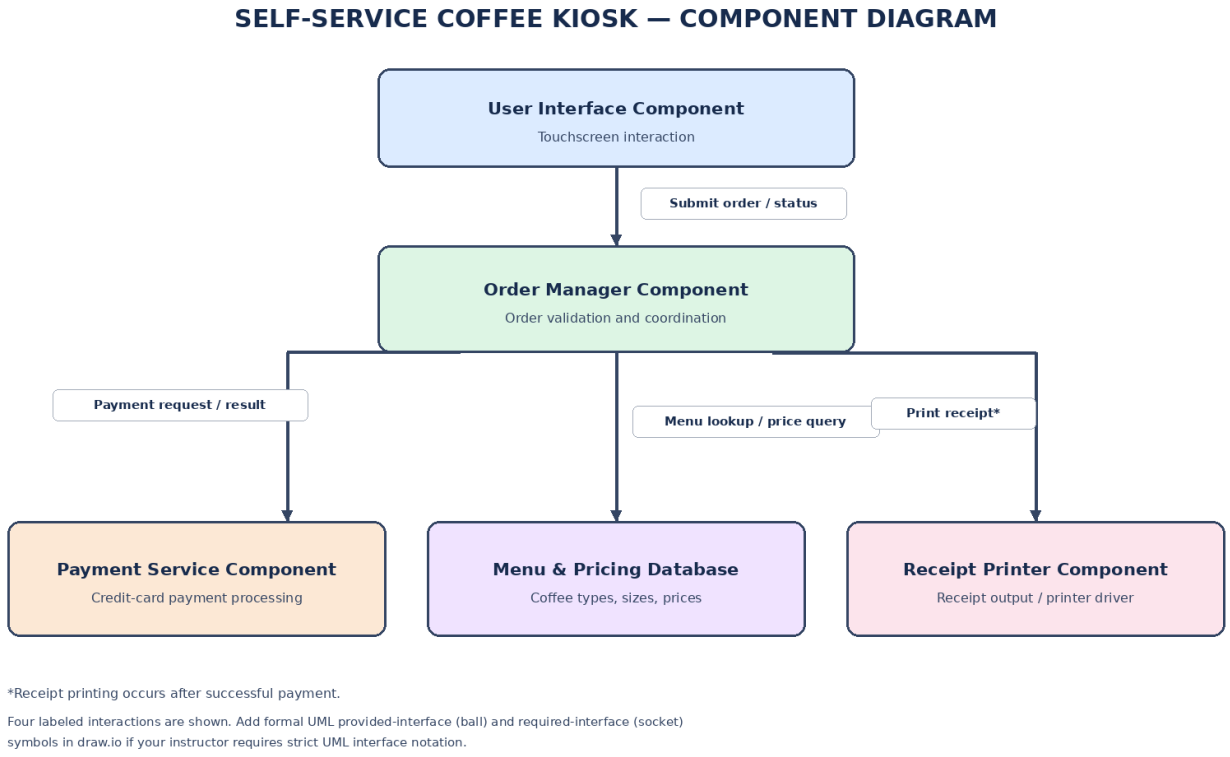
Separating menu lookup from order coordination allows menu and pricing data to be retrieved through a dedicated component and keeps the ordering flow simple. This makes performance bottlenecks easier to identify and optimize.

5. Component Identification

- **User Interface Component:** Captures touchscreen selections and displays menu, order, and payment status.
- **Order Manager Component (given in handout):** Validates coffee and size, coordinates price lookup and payment, and triggers receipt printing after successful payment.
- **Payment Service Component (given in handout):** Processes credit-card payment requests and returns the result.
- **Menu & Pricing Database Component:** Stores coffee types, available sizes, and pricing information.
- **Receipt Printer Component:** Sends order details to the receipt printer through a hardware/driver interface.

6. Component Diagram

The diagram shows the five components and four key labeled interactions. The arrows indicate the direction of the request or data flow.



UML notation requirement: the handout specifies provided interfaces (ball notation), required interfaces (socket notation), and dependencies/assembly connectors. The included diagram shows components and labeled connections; add the formal ball/socket symbols in draw.io if required for strict UML compliance.

7. Interfaces and Data Flow

- 1. User Interface → Order Manager: Submit Order / Order Status.
- 2. Order Manager → Payment Service: Payment Request / Result.
- 3. Order Manager → Menu & Pricing Database: Menu Lookup / Price Query.
- 4. Order Manager → Receipt Printer: Print Receipt after successful payment.
- External hardware connection: Receipt Printer Component → physical printer using a USB connection or printer driver.

8. Security and Performance Considerations

Security: separate the interface from payment logic, use secure communication and a trusted payment provider or terminal, and avoid unnecessary storage of card data. Performance: keep menu lookup direct and the ordering workflow simple; the modular structure helps identify bottlenecks.

9. Deliverables and Submission Checklist

☐ Component diagram exported as PNG or PDF.

☐ One-page architecture justification included in this report.

☐ Fill in student name, SRN, and section.

☐ Upload this completed document and, if required by the instructor, the standalone diagram to the Lab3 folder under the SELABS GitHub repository.

☐ Commit and push the files, then verify that they are visible in GitHub.